

Aegis RAG — Master Doc (single source of truth)

This file is the self-sufficient single source of truth for the RAG work — what it does, the backend code in detail, every command, evaluation, reset tooling, the decisions AND the reasoning behind them, and what's next. A future agent should be able to continue from this file alone. **If you change something, update this file** (and add a change-log entry at the bottom).

Remaining sibling docs (backend reference): [APISpec.md](#), [DatabaseSchema.md](#), [explanation.md](#). Frontend lives in [frontend/](#).

Last updated: 2026-06-16.

0. Quick command reference

Always run from the repo root (`cd C:\Users\nkk77\Desktop\rgbackup\sar-rag_v1`).

```
# ---- RETRIEVAL EVAL (deterministic, no Groq) ----
python eval/ir_metrics.py                # MOCK clients (testing/clients) -
default
python eval/ir_metrics.py client_0        # one mock client
python eval/ir_metrics.py --prod          # REAL clients (production/clients)
python eval/ir_metrics.py client_1 --prod # one real client

# ---- GENERATION EVAL (RAGAS-style, uses Groq judge + bge) ----
python eval/ragas_eval.py                # MOCK clients - Faithfulness + Answer
Relevancy
python eval/ragas_eval.py client_0        # one mock client
python eval/ragas_eval.py --prod          # REAL clients (production/clients)
python eval/ragas_eval.py client_1 --prod # one real client

# ---- GENERATION (needs GROQ_API_KEY in backend/.env) ----
# NOTE: these are TEST-ONLY demo scripts - always MOCK data (testing/ + client_0).
# They have NO --prod mode. Real production generation runs via the live API
(unbuilt).
python scripts/rag_smoke_test.py          # retrieval only (top-8 + cosine
distances)
python scripts/rag_generate_test.py        # RAG vs no-RAG baseline
python scripts/generate_sar_report.py      # save a SAR -> outputs/sar_*.pdf,json}
python scripts/demo_full_pipeline.py       # full flow incl. approval ->
outputs/final/

# ---- MOCK DATA: seed / reset ----
python scripts/seed_testing.py            # (re)generate the whole mock env
(policy + all JSONs)
python testing/build_policy.py             # rebuild ONLY the synthetic policy PDF
python scripts/reset.py                   # clear runtime mess (outputs + Chroma);
mock data kept
python scripts/reset.py --seed            # clear runtime + regenerate the whole
mock env
```

```
python scripts/reset.py --barebone          # delete ALL mock data (pure code)
python scripts/reset.py --barebone --seed # wipe everything, then regenerate
(fully reversible)
```

Rule of thumb: **no --prod = mock (testing/)**, **--prod = real (production/)**. Eval/smoke need no key; generation needs the Groq key.

1. What this is

Aegis AML auto-drafts Suspicious Activity Reports (SARs/STRs) for Indian fintechs/brokers. A transaction → normalized → PII-masked → 8 typology rules → if risk ≥ 75, an LLM (Groq) writes a SAR → officer reviews/approves → filed to FIU-India via goAML.

RAG makes that SAR cite the **tenant's actual AML policy** instead of the LLM's general knowledge. It's plain ("vanilla") RAG: parse → chunk → embed → store → cosine-retrieve → inject into the prompt. No reranking/hybrid/critic (we proved with metrics they aren't needed yet).

2. Architecture & flow

```
config.py / .env → embeddings.py (encoder engine, used by BOTH sides)
|
| PHASE 1 (index, once per uploaded PDF)
|   document_ingestion_service.py: parse → sections → chunks → embed →
chroma_client.store
|
| PHASE 2 (retrieve, per alert)
|   rag_retrieval_service.py: fired rules → sub-queries → embed →
chroma_client.cosine → top-8
|
| PHASE 3 (generate, per alert)
|   llm_agent.py: chunks + data → prompt → Groq → SAR (narrative + structured
JSON)
|       → (real) dashboard → officer approves → rehydrate PII → goAML JSON +
PDF → webhook
|       → (demo) simulated approval → files in outputs/final/
```

Two **shared modules** (`embeddings.py`, `chroma_client.py`) are called by both the index and retrieve sides, so the encoder and store can never drift apart. **None of retrieval uses an LLM** — bge is an encoder. Groq appears only in Phase 3.

3. Backend code (detailed)

All RAG code lives in `backend/app/services/`.

`embeddings.py` — the encoder engine

- Model: `BAAI/bge-small-en-v1.5` via sentence-transformers (SBERT). 384-dim, 512-token max, cosine. Local, free, offline. Provider-swappable (`local/openai`).
- Singleton + lazy load (key perf detail):

```
@lru_cache(maxsize=1)
def _local_model():
    from sentence_transformers import SentenceTransformer # lazy: torch
    loads only when used
    return SentenceTransformer(_LOCAL_MODEL_NAME) # loaded ONCE,
    reused
```

- Asymmetric: query gets a prefix, chunks don't:

```
QUERY_INSTRUCTION = "Represent this sentence for searching relevant passages:
"
def embed_documents(texts): return model.encode(texts,
normalize_embeddings=True).tolist()
def embed_query(text): return model.encode([QUERY_INSTRUCTION+text],
normalize_embeddings=True)[0].tolist()
```

- `count_tokens()` uses bge's own tokenizer so the chunker respects the 512 limit.

`chroma_client.py` — the vector store

```
@lru_cache(maxsize=1)
def get_chroma_client(): return chromadb.PersistentClient(path=_PERSIST_DIR) #
LOCAL files
def get_tenant_collection(tenant_id):
    return get_chroma_client().get_or_create_collection(
        name=f"tenant_{tenant_id}_docs", metadata={"hnsw:space": "cosine"}) #
per-tenant + cosine
```

`document_ingestion_service.py` — Phase 1 (index)

Constants: `CHUNK_TARGET=350`, `CHUNK_OVERLAP=60`, `MAX_CHUNK_HARD=480`.

- `parse_pdf` — PyMuPDF `get_text("dict")` → keeps font size (heading detection)
 - page number (citations). Body font size = the size covering the most text.
- `_running_lines` — GENERIC running-header/footer detection (lines repeated on $\geq 50\%$ of pages). No hardcoded client text.
- `split_sections` — split on detected headings; skip page numbers + running furniture; **drop front matter before the first numbered heading** (removes cover bank name/title/doc-control table generically).
- `_is_heading` — numbered headings always count; non-numbered must be short (≤ 60 chars) so the document title isn't mistaken for a section.

- `chunk_section` — accumulate whole sentences to ~350 tokens, emit, carry ~60 tokens of overlap into the next chunk; never cut mid-sentence. `_enforce_max_len` hard-splits any single sentence > 480 tokens (prevents silent bge truncation).
- `build_chunks` — prepend `[Context: <doc> - Section: "<heading>"]` to every chunk (so it's findable even without the keyword); attach metadata `{doc_id, filename, section_heading, chunk_index, page_number}`.
- `index_document(tenant_id, chunks)` — `embed_documents` then `collection.add`.

`rag_retrieval_service.py` — Phase 2 (retrieve)

- `RULE_TO_QUERY` — each real `rule_name` → a regulation-worded sub-query.
- We do NOT embed the JSON (masked tokens are semantic noise).
- `build_sub_queries` — one sub-query per fired rule + a fallback from txn type.
- `retrieve_regulatory_context(tenant_id, masked_payload, compliance_results, top_k=8)`:

```
collection = get_tenant_collection(tenant_id)           # ONLY this tenant's
docs
for q in sub_queries:
    res = collection.query(query_embeddings=[embed_query(q)], n_results=5)
    # merge into dict keyed by chunk_id, keep smallest distance (dedupe)
return sorted(merged.values(), key=distance)[:top_k]  # top-8; lower distance
= more relevant
```

`llm_agent.py` — Phase 3 (generate)

- `_format_regulatory_context` — formats the 8 chunks into a REGULATORY CONTEXT block.
- `build_sar_prompt` — injects that block AFTER `<<END DATA>>` (chunks are trusted; keeps them outside the untrusted-payload fence) and pins the JSON schema so `key_indicators` are always `{indicator, regulation, description}` objects.
- `_parse_response` — fence-tolerant (strips ```json fences); a JSON parse failure never discards the narrative (bug we fixed).
- `generate_sar_core(masked_payload, risk_score, model_name, retrieved_chunks)` — DB-free core (so tests run without Postgres). `generate_sar()` is the DB wrapper that persists the SARDraft and rehydrates PII for the bank-facing copy.

4. Where ChromaDB stores (LOCAL, no server)

`PersistentClient(path=...)` writes files:

```
<persist_dir>/
├─ chroma.sqlite3           # documents + metadata + ids + catalog
├─ <collection-uuid>/      # one folder per collection
│   └─ data_level0.bin      # 384-dim vectors + HNSW graph nodes
```

```

├─ header.bin / length.bin
├─ link_lists.bin
# HNSW graph edges (fast approx-NN search)

```

Default path = `backend/chroma_data/` (gitignored). Test scripts override it to a temp dir so runs don't pollute the repo. Production: point `CHROMA_PERSIST_DIR` at a stable path, or move to Chroma server / pgvector — only that setting changes.

5. Multi-tenant isolation

`tenant_id` threads through: `index_document("client_0", ...) → tenant_client_0_docs;`
`retrieve_regulatory_context("client_0", ...)` reads ONLY `tenant_client_0_docs`. Collections are physically separate — no query can cross tenants. In the live app, `tenant_id` comes from API-key auth in `ingest.py` (`alert.tenant_id`).

6. Config & secrets

`backend/app/config.py` (RAG keys): `GROQ_MODEL`, `EMBEDDING_PROVIDER`, `LOCAL_EMBEDDING_MODEL`, `CHROMA_PERSIST_DIR`, `RAG_TOP_K_CHUNKS`, `MAX_UPLOAD_FILE_SIZE_MB`. `backend/.env` (gitignored) holds the real `GROQ_API_KEY` + overrides.

7. Test data & the DUMMY client (`client_0`)

The dummy/test client is named `client_0` on purpose, so real clients start at `client_1` with no confusion.

Mock and real data are fully separated into two trees:

```

testing/                                # ALL mock data – regeneratable (python
scripts/seed_testing.py)
├─ build_policy.py                      # generates the synthetic policy PDF
├─ sources.md                          # real AML source URLs (reference)
├─ corpus/aegis_bank_aml_policy.pdf
├─ inputs/alert_structuring_intlwire.json # the generation demo input
├─ clients/client_0/                   # the DUMMY test client (named 0)
  └─ policy.pdf · config.json · alerts/01..05.json

production/                            # REAL clients – starts EMPTY
├─ corpus/ · inputs/ · clients/        # real client_1/ ... go in clients/
└─ README.md

```

All generation scripts use the `testing/` files and tenant id `client_0`. Everything under `testing/{corpus,inputs,clients}` is produced by `scripts/seed_testing.py`, so it is safe to wipe and recreate.

8. How to run

All commands are in **\$0 (Quick command reference)** at the top. Rule of thumb:

- Always run from the repo root.
- `eval/*.py` → no `--prod` = MOCK (`testing/`), `--prod` = REAL (`production/`); an optional client id narrows to one client.
- `scripts/` generation = TEST-ONLY (mock data, `client_0`); eval (`ir_metrics`, `ragas_eval`) need no key except `ragas_eval` (Groq judge); generation needs the Groq key.

9. Reset tooling — `scripts/reset.py` (the reset button)

One command to get a clean, known state at any point:

Command	Effect
<code>python scripts/reset.py</code>	Wipe runtime only: <code>outputs/</code> + all Chroma stores (repo + temp). Mock data KEPT. Safe, re-runnable any time.
<code>python scripts/reset.py --seed</code>	Wipe runtime, then regenerate the ENTIRE mock env under <code>testing/</code> (policy + demo input + <code>client_0</code> config/alerts).
<code>python scripts/reset.py --barebone</code>	Pure barebone: also deletes all mock data under <code>testing/</code> (corpus, inputs, clients). Keeps <code>testing/build_policy.py</code> + <code>sources.md</code> .
<code>python scripts/reset.py --barebone --seed</code>	Full wipe, then regenerate the whole mock env from scratch — fully reversible.

`scripts/seed_testing.py` is the single source of the mock fixtures (it embeds the demo input, `client_0` config, and the 5 eval alerts), so `--barebone` is always reversible with `--seed`. Never touches: `backend/app`, `scripts/`, `*.md`, `production/`, or PostgreSQL (`backend/clear_db.py` for SQL).

The vector store rebuilds automatically the next time you run any script, so a plain `python scripts/reset.py` mid-development is always safe.

10. Evaluation — `eval/` (deterministic, NO LLM)

Per-tenant, because the answer key (section numbers) is document-specific. Mock clients live in `testing/clients/`, real clients in `production/clients/`. The harness reads `testing/` by default, or `production/` with `--prod`.

```
testing/clients/client_0/           (production/clients/client_1/ for real)
├─ config.json    # policy_pdf + schema_preset + rule_to_section (ANSWER KEY) +
always_relevant
├─ policy.pdf     # this client's doc → its own collection
└─ alerts/*.json # this client's test alerts
```

Ground truth = stored map + live rule firing. The `rule_to_section` map (in `config.json`) is stored; per-alert relevance is derived at runtime by running the real rule engine: `relevant = {section of each fired rule} ∪ {5.1}`. No hand-labeling.

Metrics (pure arithmetic):

```

precision@k = (# relevant in top-k) / k
recall@k    = (# relevant in top-k) / (# relevant)
ndcg@k      = DCG/IDCG, DCG =  $\sum \text{rel}_i / \log_2(i+2)$ 
mrr         = 1 / (rank of first relevant)

```

Run: `python eval/ir_metrics.py [client_id]`. It loops every tenant, prints per-alert + per-tenant means + an OVERALL (mean across tenants), and saves `outputs/ir_metrics.json`.

Add a real client later: make `production/clients/client_1/` with their `policy.pdf`, `alerts/*.json`, and a `config.json` whose `rule_to_section` uses THEIR section numbers. Run `python eval/ir_metrics.py --prod` — it auto-discovers and scores against its own answer key.

11. Results (client_0, 5 alerts)

Metric	Value	Meaning
Recall@8	1.000	finds every relevant section
MRR	1.000	#1 hit always relevant
nDCG@8	0.955	relevant chunks ranked high
nDCG@3	0.765	a plausible-but-unlabeled section (5.3) sometimes enters top-3
P@8	0.425	capped (only 3-4 relevant per alert; ignore as a signal)

Verdict: retrieval is near-optimal → reranking/hybrid NOT needed yet. Re-run whenever the corpus changes; let numbers decide future layers.

12. Key decisions (at a glance — full reasoning in §17)

- Encoder: **bge-small** local (SBERT bi-encoder), 384-dim, cosine, no PII egress.
- Vector store: **ChromaDB** local, per-tenant collections.
- Chunking: ~350 tok, 60 overlap, sentence-aligned, heading sections, context prefix.
- Queries: **rule-derived sub-queries** (not the JSON); chunks injected after `<<END DATA>>`.
- Eval-first: IR (retrieval) + RAGAS (generation) → **no reranking/hybrid yet**.
- **Mock vs real** split; generation fixed to one input until real clients exist.

13. Anomaly audit (fixed 2026-06-16)

- Hardcoded footer skip → generic running-header/footer detection (multi-client).
- `MAX_CHUNK_HARD` unused → over-long sentences hard-split (no silent truncation).
- Cover/front-matter (bank name, title) → dropped before first numbered heading.
- Heading detection tightened (non-numbered must be ≤60 chars).
- Dummy client renamed `client_1` → `client_0`; all scripts use `client_0`.

- After fixes: 28 chunks (was 29), metrics unchanged → removed chunks were noise.

14. Known issues (deferred, cosmetic)

- **SAR PDF table clipping:** long cells in `render_pdf` don't word-wrap → clipped. Fix = wrap cell strings in `Paragraph()`. Data correct; rendering only.
- **LLM structured-output shape** varied; mitigated by pinned JSON schema + `normalize_structured`.
- Eval uses client `display_name` in the context-line; demo scripts use a fixed title. Both retrieve near-perfectly; minor.

15. Frontend (light — see existing specs)

The React/Vite frontend (`frontend/`) and its specs (`LandingPageSpec.md`, `MVP.md`) are separate. RAG touch-points the frontend will need: a document-upload page (Phase-1 trigger) and the SAR review/approve workspace (reads the canonical SAR record; approve fires goAML serialize → webhook). Not built yet.

16. Testing vs Production — what's code, data, and API

This is the key mental model. There are TWO different worlds:

A. Offline tooling (eval + demo scripts) — these ARE folder/command-driven:

- `testing/` (mock) vs `production/` (real) folders hold DATA only.
- `python eval/ir_metrics.py` reads `testing/`; `--prod` reads `production/`.
- The demo scripts (`scripts/*.py`) are TEST-ONLY and hardcoded to `testing/` + `client_0`. They are not how real clients are served.

B. The live API (real production path) — this is NOT folder/command-driven:

- A running FastAPI app serves ALL real tenants at once. The tenant is decided by the **API key** on each request (`ingest.py` → `alert.tenant_id`), not by a `--prod` flag or a folder.
- Data lives in PostgreSQL + per-tenant Chroma collections, keyed by `tenant_id`.
- "Test vs prod" here means separate DEPLOYMENTS/databases (a dev server vs a prod server), NOT a command flag.

The RAG code itself is environment-agnostic. `embeddings.py`, `chroma_client.py`, `document_ingestion_service.py`, `rag_retrieval_service.py`, `llm_agent.py` behave identically for any tenant, mock or real. No API issues: embeddings + Chroma are LOCAL (no external API); Groq works (only watch rate limits at high volume).

So: *whatever the pipeline does on a testing doc, it does identically on a real doc.* The `--prod` flag only switches which folder the **offline eval** reads.

Per real client you still author their `eval/.../config.json` `rule_to_section` with THEIR section numbers (data setup, not an API issue).

16a. Generation eval — RAGAS (DONE, native implementation)

`eval/ragas_eval.py` measures GENERATION quality (what IR metrics can't):

- **Faithfulness** — extract atomic claims from the SAR (Groq), check each against the grounding = (transaction data + retrieved policy chunks). Score = supported/total.
- **Answer Relevancy** — generate questions the SAR answers (Groq), embed them + the real question (bge), score = mean cosine similarity.

Result (client_0, 5 alerts): **Faithfulness $\approx$ 0.81, Answer Relevancy $\approx$ 0.70** → generation is grounded and on-topic. Combined with IR (Recall@8=1.0), the whole pipeline is validated → no reranking/hybrid justified yet.

Why native, not the ragas package: ragas 0.4.x imports a `langchain_community` path removed in LangChain 1.x → won't import on this env. We implement the same two metrics directly with Groq + bge (more robust, same methodology).

Gotchas (encoded in the script): create the Groq client LAZILY (module-level creation before torch loads segfaults); set `OMP_NUM_THREADS=1`, `TOKENIZERS_PARALLELISM=false`, `KMP_DUPLICATE_LIB_OK=TRUE` (the ragas/langchain install bumped numpy/torch → intermittent segfaults without these); faithfulness grounding MUST include the transaction data, not just policy chunks (else true transaction facts score as "unsupported").

16b. Not built yet + Next

Live-API plumbing (deferred — the demo/eval prove the logic, but the live HTTP path needs these):

- `documents.py` upload router + `TenantDocument` model — so a real client uploads their policy via API and it's indexed into THEIR Chroma collection. Until built, indexing happens only via scripts.
- Wire `retrieve_regulatory_context` into `ingest.py process_alert_background` — the live alert path currently calls `generate_sar()` WITHOUT retrieved chunks, so RAG does not yet fire on real alerts through the API (the function works; it's just not plugged in).
- Real approval → goAML → webhook delivery (the demo simulates this).
- Live `POST /ingest` integration test against the DB.

Next: RAGAS — adds generation-quality scores (LLM-judge): **faithfulness** (did the SAR invent regulation?) + **answer-relevancy** (on-topic?). IR metrics can't measure these. Plan: `pip install ragas datasets`; harness runs the pipeline over the 5 client_0 alerts, collects (`question`, `retrieved_contexts`, `answer`), points the judge LLM at Groq + embeddings at bge. Use it to confirm faithfulness and tune `top_k`. Add reranking/hybrid only if metrics later show a failure.

17. Design journey & reasoning (how we got here, and why)

This captures the *thinking* — so a future agent understands not just what exists but why each choice was made.

The build order (chronological):

1. Read the goal (cite the tenant's real AML policy, not the LLM's general knowledge) and confirmed nothing existed yet.
2. Locked the encoder/similarity/chunking decisions FIRST (they ripple through every file), before writing code.

3. Built a *synthetic* test policy we control (so we know exactly what retrieval *should* return) with thresholds matching the rule engine.
4. Wrote a mock alert, validated it against the REAL rule engine (not assumed).
5. Built the 4 services + proved retrieval with a smoke test (no DB, no Groq) — isolate failures to one layer.
6. Added generation (Groq) → saw a real cited SAR; fixed a JSON-parse bug found.
7. Built the full demo (→ goAML) for colleagues; pinned the JSON schema after seeing the LLM's output shape vary.
8. Built deterministic IR eval *because* the question arose: "do we need reranking/hybrid?" — we wanted numbers, not vibes.
9. Made eval per-tenant (real clients each have a different policy doc).
10. Split mock vs real data into **testing/** and **production/** + a one-command seed/reset, so the dummy env is reproducible and wipeable.

The conceptual reasoning (the "why" behind the design):

- **Bi-encoder, not cross-encoder.** A bi-encoder encodes chunk and query separately → store chunk vectors once → cheap cosine (**queries** + **chunks**). A cross-encoder scores every pair live (**queries** × **chunks**) and can't back a vector DB. (A reranker IS a cross-encoder on the top ~50 — a later option.)
- **bge/SBERT, not raw BERT.** Raw BERT's pooled vectors are anisotropic (collapsed cone) → cosine is meaningless. SBERT/bge is BERT *fine-tuned* so cosine reflects meaning. bge IS a BERT, trained the right way. Same model encodes both sides (mandatory — else vectors live in different spaces).
- **We embed rule-derived sub-queries, NOT the JSON.** Masked tokens + numbers are semantic noise; the policy contains none of them. The compliance rules already decided which fields matter, so each fired rule → a regulation-worded query.
- **Contextual chunking.** Prepending [**Context: doc - section**] makes a chunk findable even when its body lacks the query keyword. (The prefix text is part of the embedding — changing its wording shifts rankings slightly.)
- **Inject chunks AFTER <<END DATA>>.** Chunks are trusted policy text; keeping them outside the untrusted-payload fence preserves prompt-injection safety.
- **Eval-first, then decide.** Don't add reranking/hybrid/CRAG speculatively. IR metrics (exact, no LLM, because we own the labels) showed retrieval is near-optimal (Recall@8=1.0, nDCG@8≈0.97) → those layers aren't justified yet. RAGAS (LLM-judge) comes next for the GENERATION side, which IR can't measure.
- **PII never reaches RAG.** Masking happens before retrieval; the policy has no PII; Groq sees only tokens. Rehydration happens only at officer approval. So a TEE isn't load-bearing for RAG (it would matter only for the masker/PDF side).
- **Mock vs real separation.** **testing/** (regeneratable mock) vs **production/** (empty, real) keeps synthetic data from ever mixing with real clients — standard practice in finance where you can't test on real customer data. **seed_testing.py** embeds the fixtures so the whole mock env is reproducible from code.

Lessons / gotchas encountered:

- LLM wraps JSON in ```` <persist_dir>/ |—— chroma.sqlite3 # documents + metadata + ids + catalog
 |—— / # one folder per collection |—— data_level0.bin # 384-dim vectors + HNSW graph nodes |——
 header.bin / length.bin |—— link_lists.bin # HNSW graph edges (fast approx-NN search)

Default path = `backend/chroma\_data/` (gitignored). Test scripts override it to a temp dir so runs don't pollute the repo. Production: point `CHROMA\_PERSIST\_DIR` at a stable path, or move to Chroma server / pgvector – only that setting changes.

5. Multi-tenant isolation

`tenant\_id` threads through: `index\_document("client\_0", ...)` →
 `tenant\_client\_0\_docs`;
 `retrieve\_regulatory\_context("client\_0", ...)` reads ONLY `tenant\_client\_0\_docs`.
 Collections are physically separate – no query can cross tenants. In the live app,
 `tenant\_id` comes from API-key auth in `ingest.py` (`alert.tenant\_id`).

6. Config & secrets

`backend/app/config.py` (RAG keys): `GROQ\_MODEL`, `EMBEDDING\_PROVIDER`,
 `LOCAL\_EMBEDDING\_MODEL`, `CHROMA\_PERSIST\_DIR`, `RAG\_TOP\_K\_CHUNKS`,
 `MAX\_UPLOAD\_FILE\_SIZE\_MB`.
 `backend/.env` (gitignored) holds the real `GROQ\_API\_KEY` + overrides.

7. Test data & the DUMMY client (`client\_0`)

The dummy/test client is named **`client\_0`** on purpose, so real clients start at `client\_1` with no confusion.

****Mock and real data are fully separated into two trees:****

testing/ # ALL mock data — regeneratable (python scripts/seed_testing.py) |— build_policy.py # generates the synthetic policy PDF |— sources.md # real AML source URLs (reference) |—
 corpus/aegis_bank_aml_policy.pdf |— inputs/alert_structuring_intlwire.json # the generation demo input
 |— clients/client_0/ # the DUMMY test client (named 0) |— policy.pdf · config.json · alerts/01..05.json
 production/ # REAL clients — starts EMPTY |— corpus/ · inputs/ · clients/ # real client_1/ ... go in clients/
 |— README.md

All generation scripts use the `testing/` files and tenant id `client\_0`.
 Everything under `testing/{corpus,inputs,clients}` is produced by
 `scripts/seed\_testing.py`, so it is safe to wipe and recreate.

8. How to run

All commands are in `**$0 (Quick command reference)**` at the top. Rule of thumb:

- Always run from the repo root.
- ``eval/*.py`` → no ``--prod`` = MOCK (``testing/``), ``--prod`` = REAL (``production/``); an optional client id narrows to one client.
- ``scripts/`` generation = TEST-ONLY (mock data, ``client_0``); `eval (`ir_metrics`, `ragas_eval`)` need no key except ``ragas_eval`` (Groq judge); generation needs the Groq key.

9. Reset tooling – ``scripts/reset.py`` (the reset button)

One command to get a clean, known state at any point:

Command	Effect
<code>`python scripts/reset.py`</code>	**Wipe runtime only:** <code>`outputs/`</code> + all Chroma stores (repo + temp). Mock data KEPT. Safe, re-runnable any time.
<code>`python scripts/reset.py --seed`</code>	Wipe runtime, then **regenerate the ENTIRE mock env** under <code>`testing/`</code> (policy + demo input + <code>`client_0`</code> config/alerts).
<code>`python scripts/reset.py --barebone`</code>	**Pure barebone:** also deletes all mock data under <code>`testing/`</code> (corpus, inputs, clients). Keeps <code>`testing/build_policy.py`</code> + <code>`sources.md`</code> .
<code>`python scripts/reset.py --barebone --seed`</code>	Full wipe, then regenerate the whole mock env from scratch – fully reversible.

``scripts/seed_testing.py`` is the single source of the mock fixtures (it embeds the demo input, ``client_0`` config, and the 5 eval alerts), so ``--barebone`` is always reversible with ``--seed``. Never touches: ``backend/app``, ``scripts/``, ``*.md``, ``production/``, or PostgreSQL (``backend/clear_db.py`` for SQL).

> The vector store rebuilds automatically the next time you run any script, so a plain ``python scripts/reset.py`` mid-development is always safe.

10. Evaluation – ``eval/`` (deterministic, NO LLM)

Per-tenant, because the answer key (section numbers) is document-specific. Mock clients live in ``testing/clients/``, real clients in ``production/clients/``. The harness reads ``testing/`` by default, or ``production/`` with ``--prod``.

testing/clients/client_0/ (production/clients/client_1/ for real) |—— config.json # policy_pdf + schema_preset + rule_to_section (ANSWER KEY) + always_relevant |—— policy.pdf # this client's doc → its own collection |—— alerts/*.json # this client's test alerts

****Ground truth = stored map + live rule firing.**** The ``rule_to_section`` map (in ``config.json``) is stored; per-alert relevance is derived at runtime by running the real rule engine: ``relevant = {section of each fired rule} U {5.1}``. No hand-

```
labeling.

**Metrics (pure arithmetic):**
```python
precision@k = (# relevant in top-k) / k
recall@k = (# relevant in top-k) / (# relevant)
ndcg@k = DCG/IDCG, DCG = $\sum \text{rel}_i / \log_2(i+2)$
mrr = 1 / (rank of first relevant)
```

Run: `python eval/ir_metrics.py [client_id]`. It loops every tenant, prints per-alert + per-tenant means + an OVERALL (mean across tenants), and saves `outputs/ir_metrics.json`.

**Add a real client later:** make `production/clients/client_1/` with their `policy.pdf`, `alerts/*.json`, and a `config.json` whose `rule_to_section` uses THEIR section numbers. Run `python eval/ir_metrics.py --prod` — it auto-discovers and scores against its own answer key.

---

## 11. Results (client\_0, 5 alerts)

Metric	Value	Meaning
Recall@8	<b>1.000</b>	finds every relevant section
MRR	<b>1.000</b>	#1 hit always relevant
nDCG@8	<b>0.955</b>	relevant chunks ranked high
nDCG@3	0.765	a plausible-but-unlabeled section (5.3) sometimes enters top-3
P@8	0.425	capped (only 3-4 relevant per alert; ignore as a signal)

**Verdict:** retrieval is near-optimal → reranking/hybrid NOT needed yet. Re-run whenever the corpus changes; let numbers decide future layers.

---

## 12. Key decisions (at a glance — full reasoning in §17)

- Encoder: **bge-small** local (SBERT bi-encoder), 384-dim, cosine, no PII egress.
- Vector store: **ChromaDB** local, per-tenant collections.
- Chunking: ~350 tok, 60 overlap, sentence-aligned, heading sections, context prefix.
- Queries: **rule-derived sub-queries** (not the JSON); chunks injected after `<<END DATA>>`.
- Eval-first: IR (retrieval) + RAGAS (generation) → **no reranking/hybrid yet**.
- **Mock vs real** split; generation fixed to one input until real clients exist.

---

## 13. Anomaly audit (fixed 2026-06-16)

- Hardcoded footer skip → generic running-header/footer detection (multi-client).
- `MAX_CHUNK_HARD` unused → over-long sentences hard-split (no silent truncation).
- Cover/front-matter (bank name, title) → dropped before first numbered heading.
- Heading detection tightened (non-numbered must be ≤60 chars).

- Dummy client renamed `client_1` → `client_0`; all scripts use `client_0`.
- After fixes: 28 chunks (was 29), metrics unchanged → removed chunks were noise.

## 14. Known issues (deferred, cosmetic)

- **SAR PDF table clipping:** long cells in `render_pdf` don't word-wrap → clipped. Fix = wrap cell strings in `Paragraph()`. Data correct; rendering only.
- **LLM structured-output shape** varied; mitigated by pinned JSON schema + `normalize_structured`.
- Eval uses client `display_name` in the context-line; demo scripts use a fixed title. Both retrieve near-perfectly; minor.

## 15. Frontend (light — see existing specs)

The React/Vite frontend (`frontend/`) and its specs (`LandingPageSpec.md`, `MVP.md`) are separate. RAG touch-points the frontend will need: a document-upload page (Phase-1 trigger) and the SAR review/approve workspace (reads the canonical SAR record; approve fires goAML serialize → webhook). Not built yet.

## 16. Testing vs Production — what's code, data, and API

This is the key mental model. There are TWO different worlds:

**A. Offline tooling (eval + demo scripts)** — these ARE folder/command-driven:

- `testing/` (mock) vs `production/` (real) folders hold DATA only.
- `python eval/ir_metrics.py` reads `testing/`; `--prod` reads `production/`.
- The demo scripts (`scripts/*.py`) are TEST-ONLY and hardcoded to `testing/` + `client_0`. They are not how real clients are served.

**B. The live API (real production path)** — this is NOT folder/command-driven:

- A running FastAPI app serves ALL real tenants at once. The tenant is decided by the **API key** on each request (`ingest.py` → `alert.tenant_id`), not by a `--prod` flag or a folder.
- Data lives in PostgreSQL + per-tenant Chroma collections, keyed by `tenant_id`.
- "Test vs prod" here means separate DEPLOYMENTS/databases (a dev server vs a prod server), NOT a command flag.

**The RAG code itself is environment-agnostic.** `embeddings.py`, `chroma_client.py`, `document_ingestion_service.py`, `rag_retrieval_service.py`, `llm_agent.py` behave identically for any tenant, mock or real. No API issues: embeddings + Chroma are LOCAL (no external API); Groq works (only watch rate limits at high volume).

So: *whatever the pipeline does on a testing doc, it does identically on a real doc.* The `--prod` flag only switches which folder the **offline eval** reads.

**Per real client you still author** their `eval/.../config.json` `rule_to_section` with THEIR section numbers (data setup, not an API issue).

## 16a. Generation eval — RAGAS (DONE, native implementation)

`eval/ragas_eval.py` measures GENERATION quality (what IR metrics can't):

- **Faithfulness** — extract atomic claims from the SAR (Groq), check each against the grounding = (transaction data + retrieved policy chunks). Score = supported/total.
- **Answer Relevancy** — generate questions the SAR answers (Groq), embed them + the real question (bge), score = mean cosine similarity.

Result (client\_0, 5 alerts): **Faithfulness  $\approx$  0.81, Answer Relevancy  $\approx$  0.70** → generation is grounded and on-topic. Combined with IR (Recall@8=1.0), the whole pipeline is validated → no reranking/hybrid justified yet.

**Why native, not the ragas package:** ragas 0.4.x imports a `langchain_community` path removed in LangChain 1.x → won't import on this env. We implement the same two metrics directly with Groq + bge (more robust, same methodology).

**Gotchas (encoded in the script):** create the Groq client LAZILY (module-level creation before torch loads segfaults); set `OMP_NUM_THREADS=1, TOKENIZERS_PARALLELISM=false, KMP_DUPLICATE_LIB_OK=TRUE` (the ragas/langchain install bumped numpy/torch → intermittent segfaults without these); faithfulness grounding MUST include the transaction data, not just policy chunks (else true transaction facts score as "unsupported").

## 16b. Not built yet + Next

**Live-API plumbing (deferred — the demo/eval prove the logic, but the live HTTP path needs these):**

- `documents.py` upload router + `TenantDocument` model — so a real client uploads their policy via API and it's indexed into THEIR Chroma collection. Until built, indexing happens only via scripts.
- Wire `retrieve_regulatory_context` into `ingest.py process_alert_background` — the live alert path currently calls `generate_sar()` WITHOUT retrieved chunks, so RAG does not yet fire on real alerts through the API (the function works; it's just not plugged in).
- Real approval → goAML → webhook delivery (the demo simulates this).
- Live `POST /ingest` integration test against the DB.

**Next: RAGAS** — adds generation-quality scores (LLM-judge): **faithfulness** (did the SAR invent regulation?) + **answer-relevancy** (on-topic?). IR metrics can't measure these. Plan: `pip install ragas datasets`; harness runs the pipeline over the 5 client\_0 alerts, collects (`question`, `retrieved_contexts`, `answer`), points the judge LLM at Groq + embeddings at bge. Use it to confirm faithfulness and tune `top_k`. Add reranking/hybrid only if metrics later show a failure.

## 17. Design journey & reasoning (how we got here, and why)

This captures the *thinking* — so a future agent understands not just what exists but why each choice was made.

**The build order (chronological):**

1. Read the goal (cite the tenant's real AML policy, not the LLM's general knowledge) and confirmed nothing existed yet.

2. Locked the encoder/similarity/chunking decisions FIRST (they ripple through every file), before writing code.
3. Built a *synthetic* test policy we control (so we know exactly what retrieval *should* return) with thresholds matching the rule engine.
4. Wrote a mock alert, validated it against the REAL rule engine (not assumed).
5. Built the 4 services + proved retrieval with a smoke test (no DB, no Groq) — isolate failures to one layer.
6. Added generation (Groq) → saw a real cited SAR; fixed a JSON-parse bug found.
7. Built the full demo (→ goAML) for colleagues; pinned the JSON schema after seeing the LLM's output shape vary.
8. Built deterministic IR eval *because* the question arose: "do we need reranking/hybrid?" — we wanted numbers, not vibes.
9. Made eval per-tenant (real clients each have a different policy doc).
10. Split mock vs real data into **testing/** and **production/** + a one-command seed/reset, so the dummy env is reproducible and wipeable.

### The conceptual reasoning (the "why" behind the design):

- **Bi-encoder, not cross-encoder.** A bi-encoder encodes chunk and query separately → store chunk vectors once → cheap cosine (**queries** + **chunks**). A cross-encoder scores every pair live (**queries** × **chunks**) and can't back a vector DB. (A reranker IS a cross-encoder on the top ~50 — a later option.)
- **bge/SBERT, not raw BERT.** Raw BERT's pooled vectors are anisotropic (collapsed cone) → cosine is meaningless. SBERT/bge is BERT *fine-tuned* so cosine reflects meaning. bge IS a BERT, trained the right way. Same model encodes both sides (mandatory — else vectors live in different spaces).
- **We embed rule-derived sub-queries, NOT the JSON.** Masked tokens + numbers are semantic noise; the policy contains none of them. The compliance rules already decided which fields matter, so each fired rule → a regulation-worded query.
- **Contextual chunking.** Prepending [**Context: doc - section**] makes a chunk findable even when its body lacks the query keyword. (The prefix text is part of the embedding — changing its wording shifts rankings slightly.)
- **Inject chunks AFTER <<END DATA>>.** Chunks are trusted policy text; keeping them outside the untrusted-payload fence preserves prompt-injection safety.
- **Eval-first, then decide.** Don't add reranking/hybrid/CRAG speculatively. IR metrics (exact, no LLM, because we own the labels) showed retrieval is near-optimal (Recall@8=1.0, nDCG@8≈0.97) → those layers aren't justified yet. RAGAS (LLM-judge) comes next for the GENERATION side, which IR can't measure.
- **PII never reaches RAG.** Masking happens before retrieval; the policy has no PII; Groq sees only tokens. Rehydration happens only at officer approval. So a TEE isn't load-bearing for RAG (it would matter only for the masker/PDF side).
- **Mock vs real separation.** **testing/** (regeneratable mock) vs **production/** (empty, real) keeps synthetic data from ever mixing with real clients — standard practice in finance where you can't test on real customer data. **seed\_testing.py** embeds the fixtures so the whole mock env is reproducible from code.

### Lessons / gotchas encountered:

- LLM wraps JSON in ``json fences → parser must strip them and never discard the narrative on a JSON failure.
- LLM structured-output shape varies → pin the schema in the prompt + normalize on approval.

- Hardcoding one client's footer text breaks multi-client → detect running headers/footers generically.
- An unused `MAX_CHUNK_HARD` meant a giant sentence could silently truncate → hard-split.
- The context-line `doc_title` affects embeddings → eval vs demo can differ slightly.

## 18. Change log

- 2026-06-16: Built vanilla RAG (Phases 1-3), demo, per-tenant IR eval. Anomaly audit + fixes. Renamed dummy client to `client_0`. Added `scripts/reset.py`. Consolidated docs into this file (replaced RAG\_TECHNICAL\_NOTES / RAG\_WALKTHROUGH / PRE\_RAGAS).
- 2026-06-16 (later 4): RAGAS done — `eval/ragas_eval.py` (native, Groq judge + bge). Faithfulness≈0.81, Answer Relevancy≈0.70. ragas pip pkg incompatible w/ LangChain 1.x; segfault fixes (lazy Groq + OMP/tokenizer env guards); faithfulness grounds on transaction data + chunks. Pipeline fully validated; no reranking/hybrid needed.
- 2026-06-16 (later 3): User deleted the older root docs (BuildOrder/LandingPageSpec/MVP/PRD/RAG\_HANDOFF/README). Made THIS file fully self-sufficient: added §17 "Design journey & reasoning" (the thinking + lessons), ensured §0 has every command. Remaining root md: APISpec, DatabaseSchema, RAG\_MASTER, explanation.
- 2026-06-16 (later 2): Added §0 quick command reference and §16 "Testing vs Production" mental model (offline tooling is folder/flag-driven; the live API is tenant-keyed, not folder-driven; RAG code is environment-agnostic).
- 2026-06-16 (later): Split mock vs real data into `testing/` (regeneratable mock) and `production/` (empty, real). Moved policy→`testing/corpus`, input→`testing/inputs`, eval client\_0→`testing/clients`; removed `aml_corpus/`, `mock_inputs/`, `eval/tenants/`. Added `scripts/seed_testing.py` (embeds all mock fixtures, fully regenerates). `reset.py --seed` now rebuilds the whole mock env; `--barebone` wipes `testing/` data. `eval/ir_metrics.py --prod` evaluates `production/` clients. Verified full wipe→regenerate→eval cycle.
- 2026-06-17: Added §19 "Full walkthrough" (read-it-start-to-finish narrative: goal → concepts → code → eval). RAGAS made per-client + `--prod`. Slimmed §8/§12 to remove duplication.

## 19. Full walkthrough (read start-to-finish: goal → concepts → code → eval)

A single narrative explainer. The earlier sections are the structured reference; this is the teaching version for someone learning the whole thing cold.

### 1. The goal

Aegis writes SARs with Groq. Without RAG the LLM uses general knowledge and can't cite the tenant's policy. RAG fetches the relevant policy slices and feeds them to the LLM, so the SAR cites e.g. "Section 4.1" instead of vague prose. RAG = Retrieve relevant policy → Augment the prompt → Generate the cited SAR.

### 2. Core concepts

- **Embeddings:** text → 384-number vector; similar meaning → similar direction; compared by **cosine similarity** (angle; small angle = similar).
- **Bi-encoder vs cross-encoder:** a bi-encoder encodes chunk and query SEPARATELY → store chunk vectors once → cheap cosine (`chunks + queries`). A cross-encoder scores each (query,chunk) PAIR live

(**chunks × queries**) and can't back a vector DB. (A reranker is a cross-encoder on the top ~50 — a later option.)

- **BERT vs SBERT:** raw BERT vectors are anisotropic (cosine meaningless); bge is BERT fine-tuned so cosine reflects meaning. bge IS a BERT, trained right.
- **Asymmetric:** prefix the QUERY only ("Represent this sentence for searching relevant passages: "); chunks raw.
- **Contextual chunking:** prepend [**Context: doc - section**] to each chunk → it's findable even without the keyword.
- **Don't embed the JSON:** masked tokens/numbers are noise → embed rule-derived sub-queries instead.

### 3. The pipeline + code (3 phases)

Shared engine: **embeddings.py** (make vectors) + **chroma\_client.py** (per-tenant store).

```
embeddings.py – one model, loaded once, used by BOTH sides
@lru_cache(maxsize=1)
def _local_model():
 from sentence_transformers import SentenceTransformer
 return SentenceTransformer("BAAI/bge-small-en-v1.5")
def embed_documents(texts): return _local_model().encode(texts,
normalize_embeddings=True).tolist()
def embed_query(text): return _local_model().encode(["Represent this sentence for
searching relevant passages: "+text], normalize_embeddings=True)[0].tolist()

chroma_client.py – per-tenant isolation + cosine
def get_tenant_collection(tid):
 return chromadb.PersistentClient(path=_PERSIST_DIR).get_or_create_collection(
 name=f"tenant_{tid}_docs", metadata={"hnsw:space": "cosine"})
```

#### Phase 1 (index) — document\_ingestion\_service.py: PyMuPDF parse (keep font size

- page) → split on headings → ~350-token sentence-aligned chunks w/ 60 overlap → prepend context line → embed → **collection.add(...)**. **Phase 2 (retrieve) — rag\_retrieval\_service.py:** fired rules → **RULE\_TO\_QUERY** sub-queries → embed each → **collection.query(n\_results=5)** → merge/dedupe → top-8. **Phase 3 (generate) — llm\_agent.py:** inject chunks AFTER <<END DATA>> (trusted text) → Groq → parse narrative + structured JSON → rehydrate PII at approval.

### 4. Where data lives

ChromaDB local files: **chroma.sqlite3** (docs+metadata) + per-collection HNSW binaries. No server. Tests use a temp dir.

### 5. Multi-tenant

**tenant\_id** keys the collection name, so **index\_document("client\_0",...)** and **retrieve\_regulatory\_context("client\_0",...)** only ever touch **tenant\_client\_0\_docs**. No query crosses tenants. Live app gets **tenant\_id** from the API key.

### 6. Evaluation (both halves)

- **Retrieval — `ir_metrics.py` (no LLM):** we own the labels (rule → section in config.json), so Precision@k/Recall@k/nDCG/MRR are pure arithmetic. Result: Recall@8=1.0, nDCG@8=0.98, MRR=1.0.
- **Generation — `ragas_eval.py` (Groq judge + bge):** Faithfulness (Groq extracts claims, checks each vs transaction-data+chunks) = 0.81; Answer Relevancy (Groq makes questions from the SAR, bge cosine-compares to the real question) = 0.70.

## 7. Test infrastructure

`testing/` (regeneratable mock) vs `production/` (empty, real). Dummy = `client_0.scripts/reset.py` (+ `--seed/--barebone`) and `scripts/seed_testing.py` make the mock env fully wipeable and reproducible.

## 8. Outcome

**Vanilla RAG was sufficient — proven by metrics, not assumed.** No reranking/ hybrid/CRAG needed (retrieval perfect, generation grounded). Caveat: re-evaluate when real multi-doc corpora arrive. Remaining work = wire RAG into the live app (API endpoints + frontend); the RAG brain itself is done.

**One line:** embeddings make vectors → ingestion fills Chroma (P1) → retrieval cosine-searches it (P2) → `llm_agent` writes the cited SAR (P3) → `ir_metrics` proves retrieval + `ragas_eval` proves generation → all per-tenant, mock-vs-real, resettable.